Ardunic
build anything

Automated Silhouette Component Counter

An Edge-Computed Machine Vision Workstation on the ESP32-CAM

By Morteda Hokman



Outlines:

- Project Overview
- The Problems And The Solutions
- Components & Tools
- Circuit Diagram
- Hardware Implementation
- Software Process
- Software Implementation
- Challenges & Hardware Debugging
- Future Improvements

Project Overview

- This project is an automated machine-vision workstation designed to eliminate the time-consuming bottleneck of manually counting small electronic components (resistors, ICs, connectors) for bulk customer orders
- To overcome traditional camera issues like metallic glare and overlapping parts, the system uses a bottom-lit LED stage to create high-contrast black silhouettes, combined with a 4-corner mechanical vibration array that physically shakes clumped components apart
- Powered entirely by a single ESP32-CAM, it processes these silhouettes natively at 30 frames per second, instantly streaming an accurate, real-time count to the operator's web browser to ensure fast and flawless order fulfilment

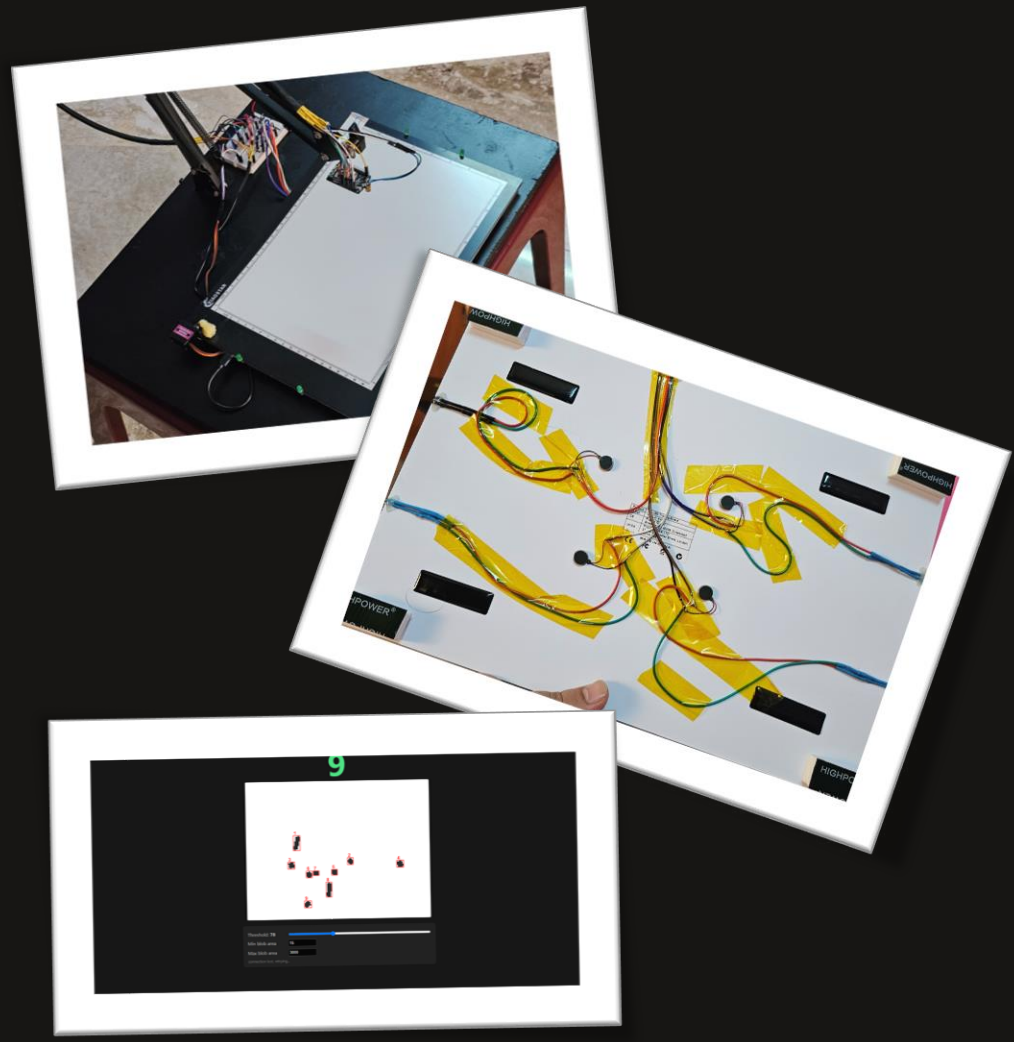
The Problems

- **Slow Fulfillment**: Counting dozens of small parts (resistors, ICs, connectors) by hand causes long delays in order packing
- **Human Error:** Visual fatigue leads to under-counting (upsetting customers) or over-counting (direct inventory loss)
- **Standard Camera Failures**: Metallic pins cause glare, overhead lighting creates shadows, and dumped parts overlap, ruining standard vision algorithms

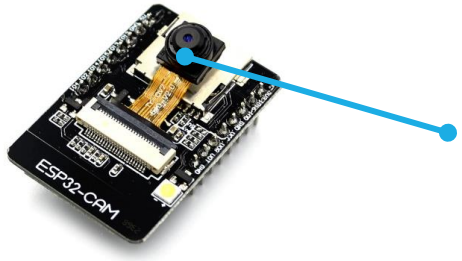


The Solutions

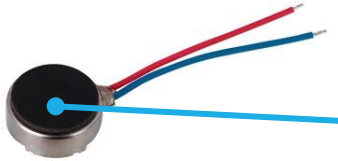
- **Optical Backlighting (Zero Glare):** A bottom-lit LED stage turns components into razor-sharp dark silhouettes, eliminating shadows and color noise
- **Mechanical Agitation (Zero Overlap):** A 4-corner vibration motor array physically shakes clumped parts apart so they can be counted individually
- **Instant Edge Counting (20–30 FPS):** The ESP32-CAM processes the separated silhouettes in real-time, giving the operator an exact count on any web browser instantly



Components & Tools



ESP32-CAM



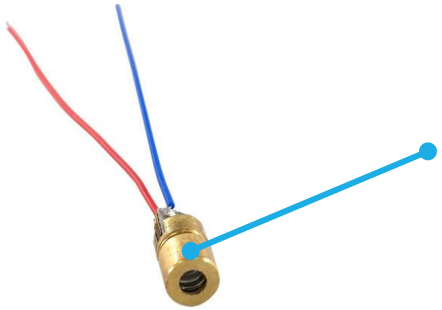
Motor Vibrator



MG90S Servo Motor



5mm LED



650nm Laser



2N 2222A NPN Transistor

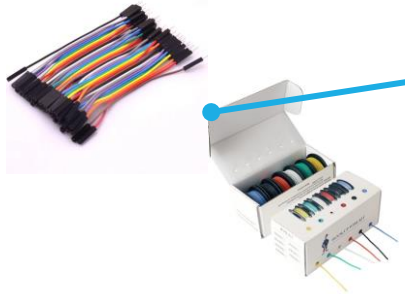
Components & Tools



5V 3A Charger Adapter



3,3 Voltage Regulator AMS1117



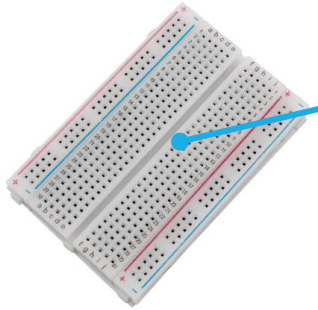
Wires



Heat Shrinks

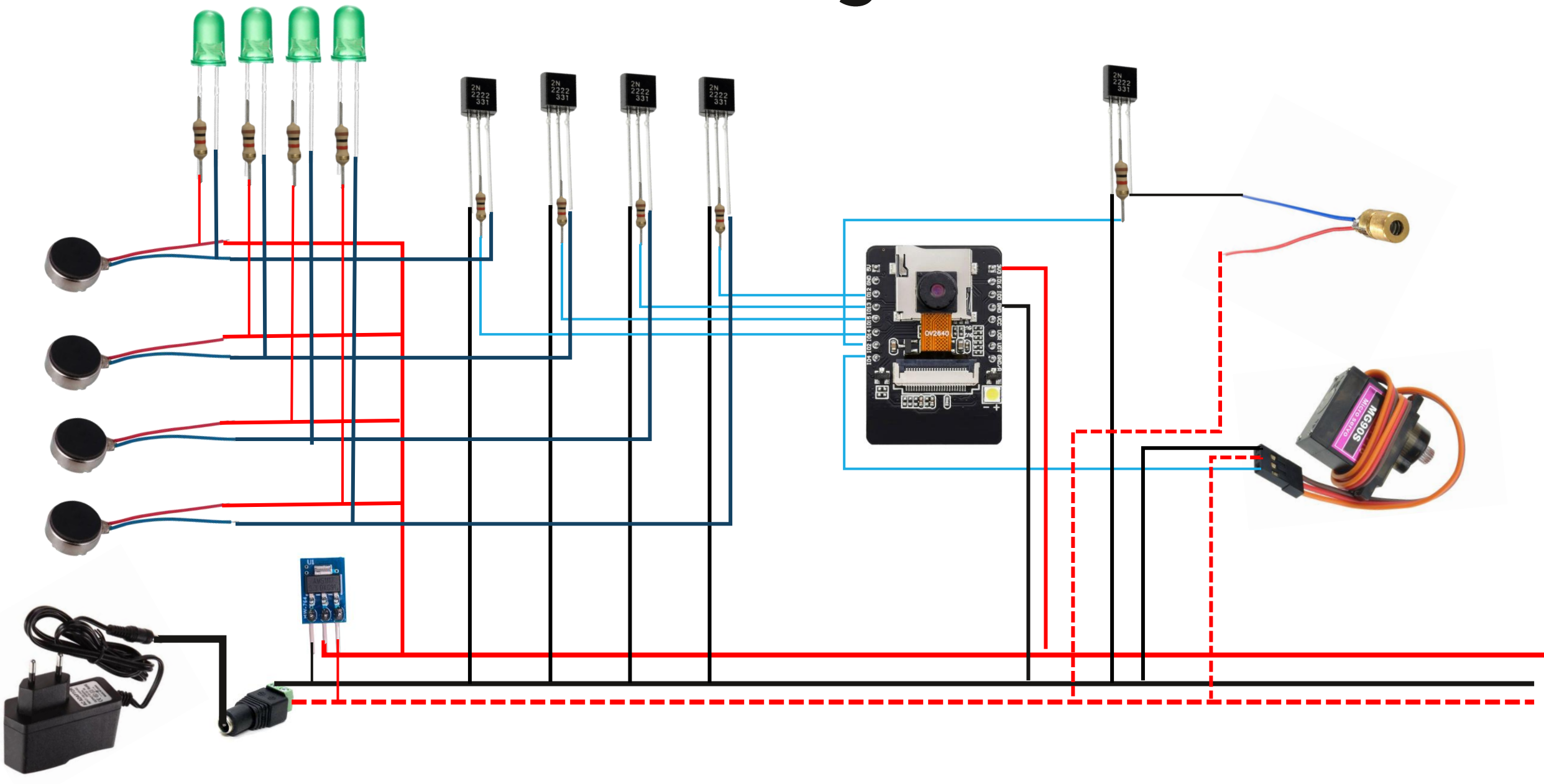


CCTV Female DC Power Connector

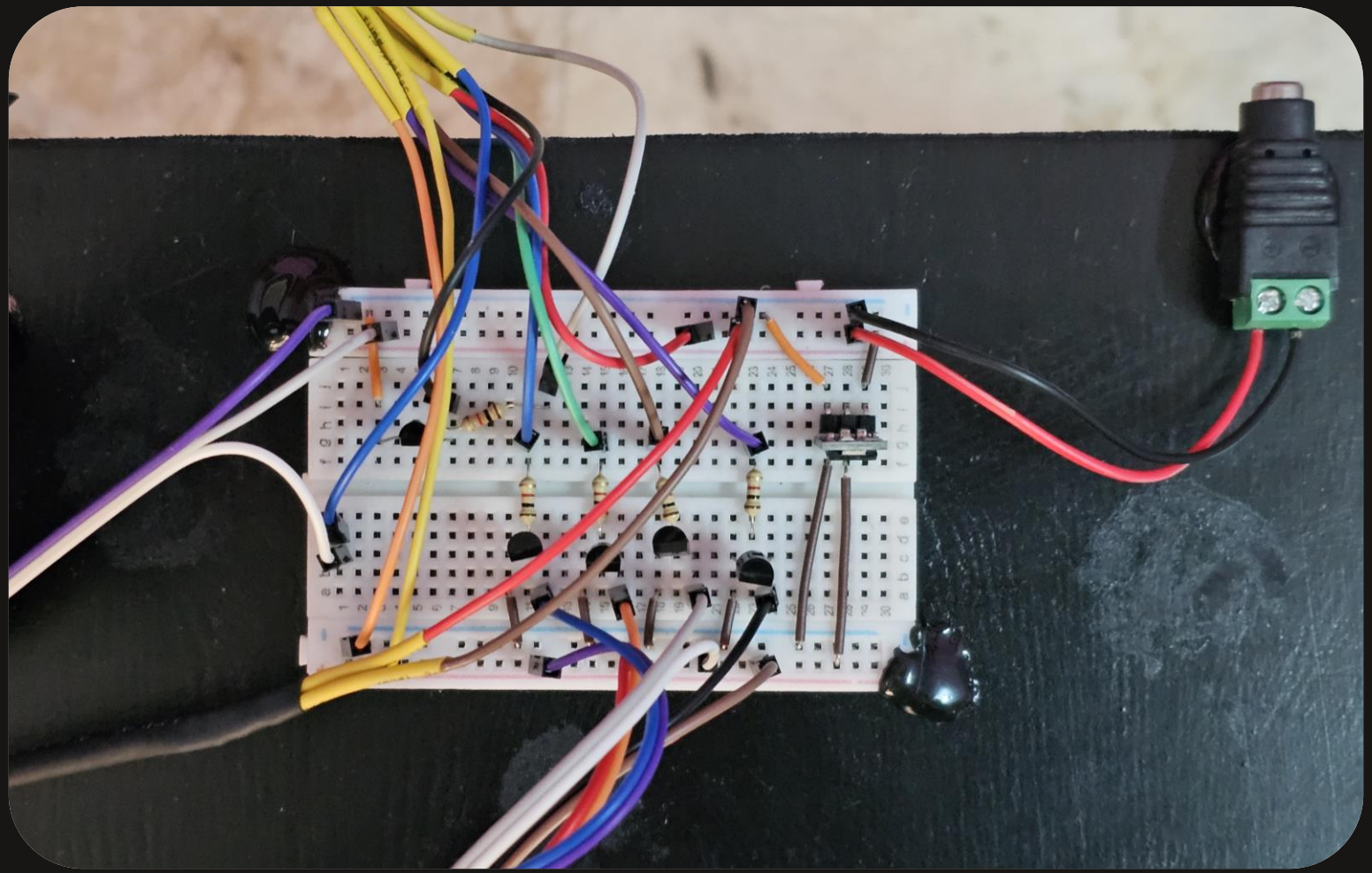


400P breadboard

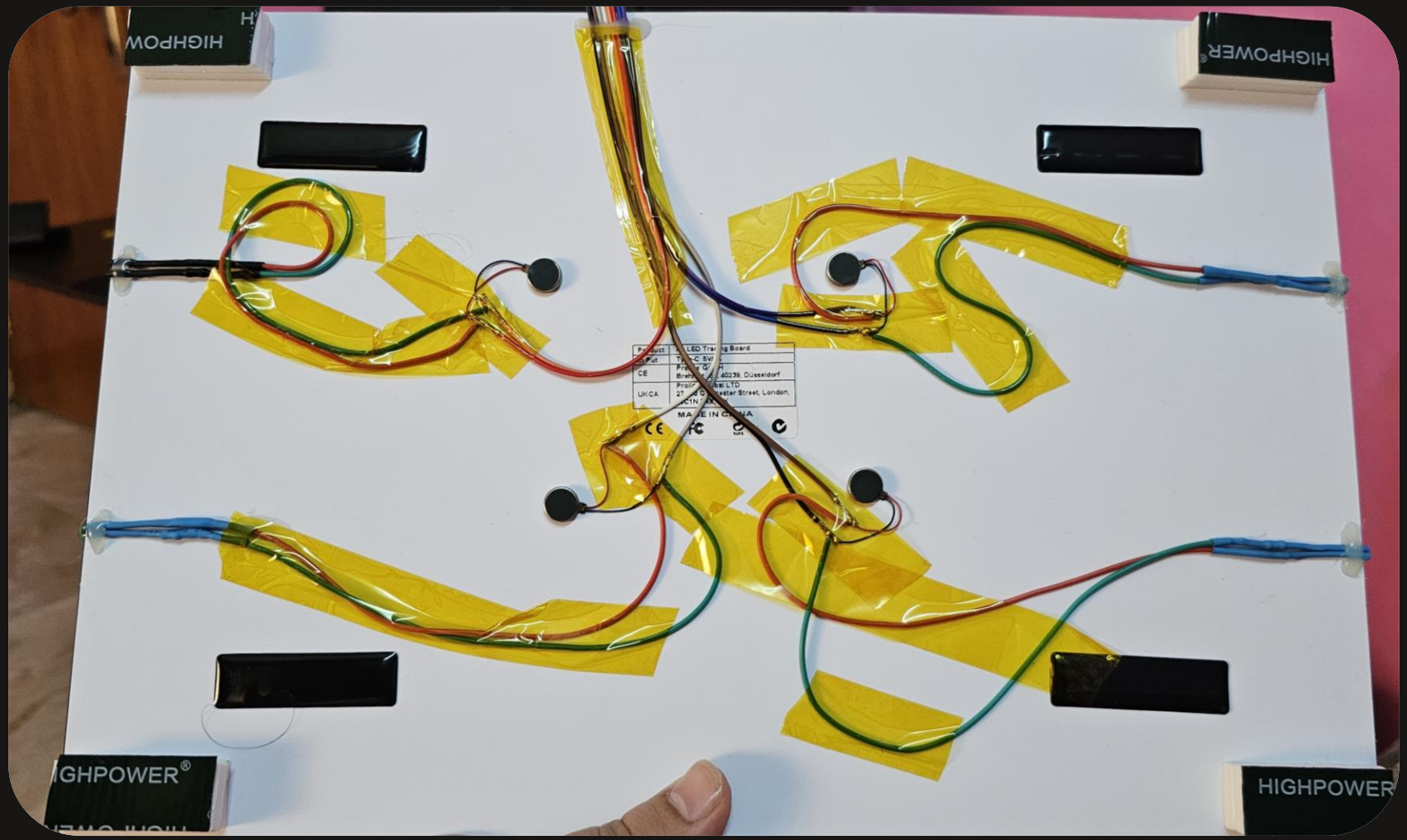
Circuit Diagram



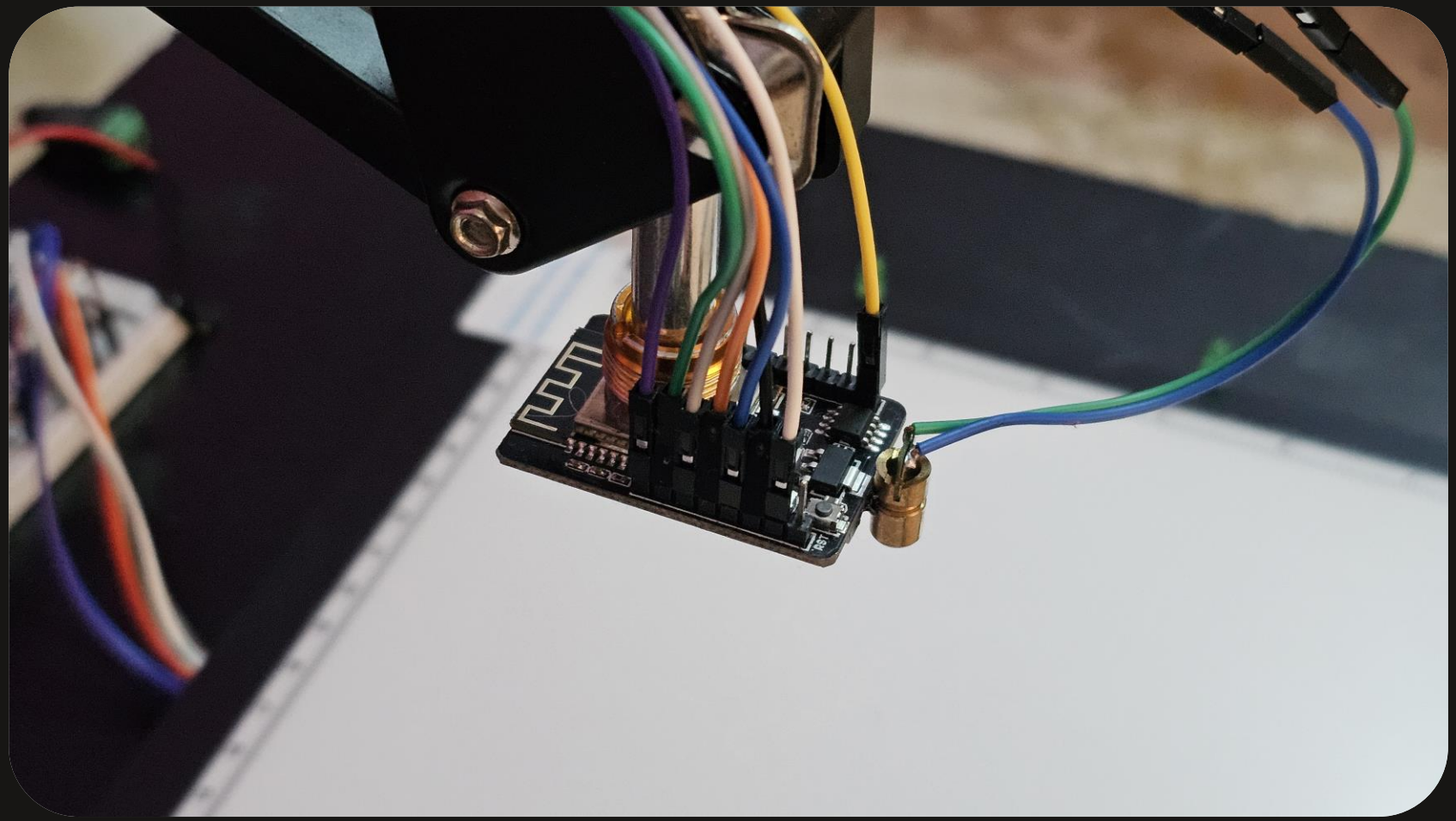
Hardware Implementation



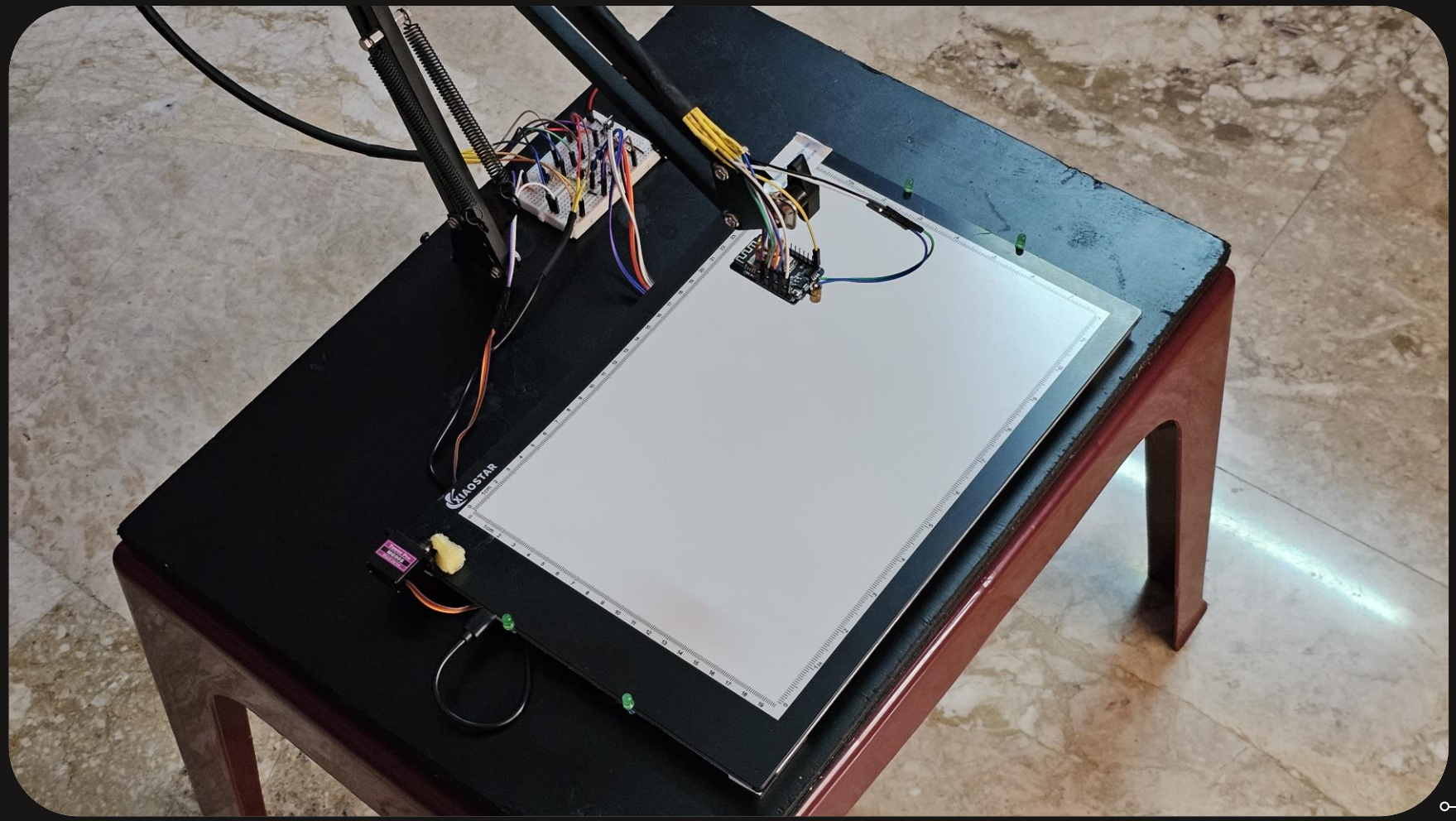
Hardware Implementation



Hardware Implementation



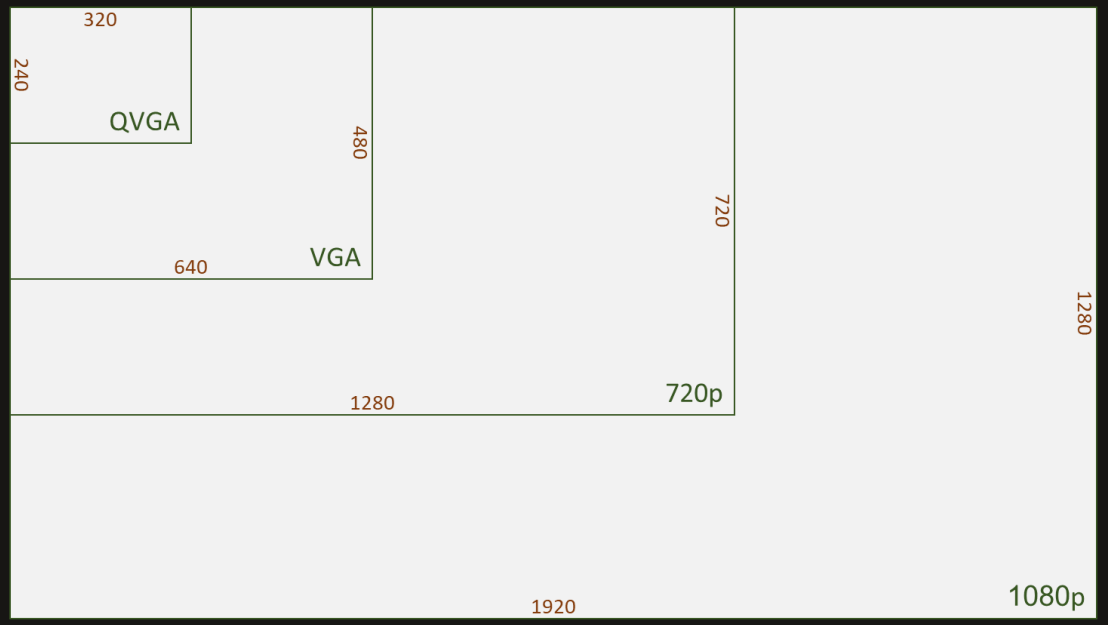
Hardware Implementation

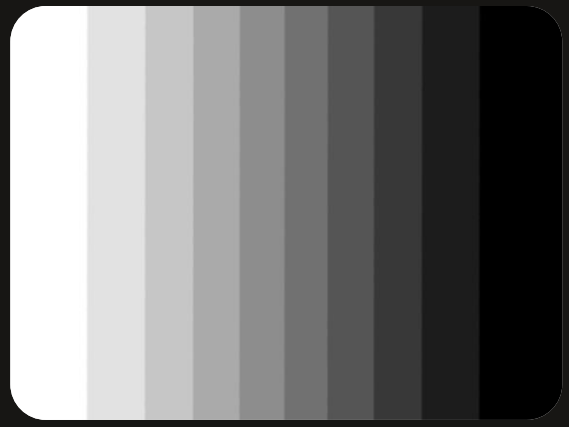


Software Process

- **Raw Grayscale Capture**: The OV2640 camera captures a 160x120 (QQVGA) grayscale image. Automatic white balance is disabled to maintain a fixed exposure against the bright LED background.

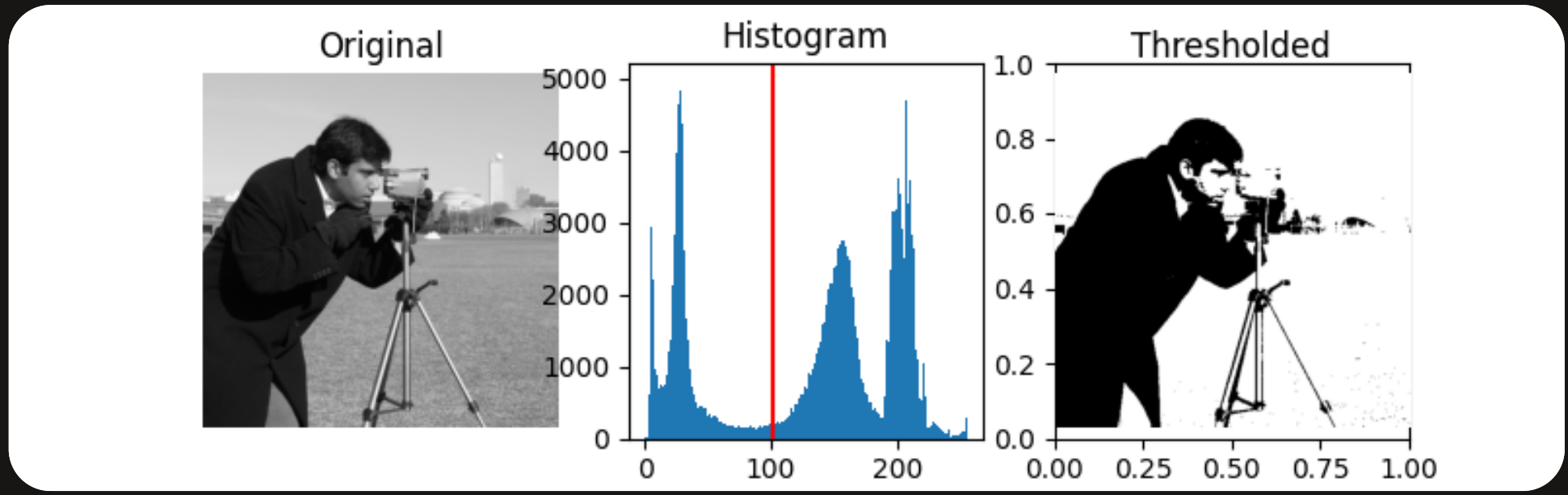






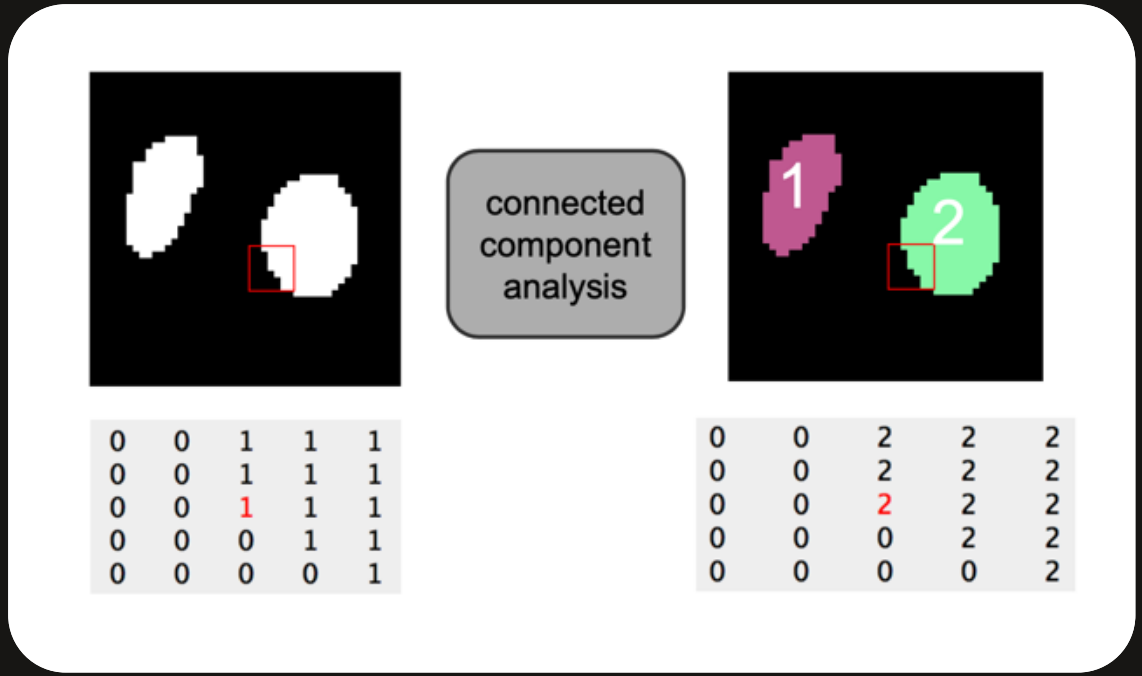
Software Process

- **Binary Thresholding**: The software evaluates every pixel against an adjustable brightness threshold. Dark pixels (the silhouettes) are classified as Foreground (1), and bright pixels (the light pad) are classified as Background (0).



Software Process

- **Connected-Component Labeling (8-Way BFS):** The algorithm scans the binary map. When it finds a foreground pixel, it launches a Breadth-First Search (BFS) using a pre-allocated ring queue to group all connecting pixels in 8 directions, forming a complete "blob" and determining its bounding box.



Software Process

- **Size & Noise Filtering:** To guarantee an accurate count, the detected blobs are filtered mathematically:
  - Minimum Area (15 px): Rejects dust specks, sensor noise, or scratches on the light pad.
  - Maximum Area (3000 px): Rejects large shadows, tools, or the operator's hands entering the frame

- **Output Generation**: The validated blobs increment the final component count. The coordinates for the red bounding boxes and the compressed binary image are then packaged for real-time streaming.

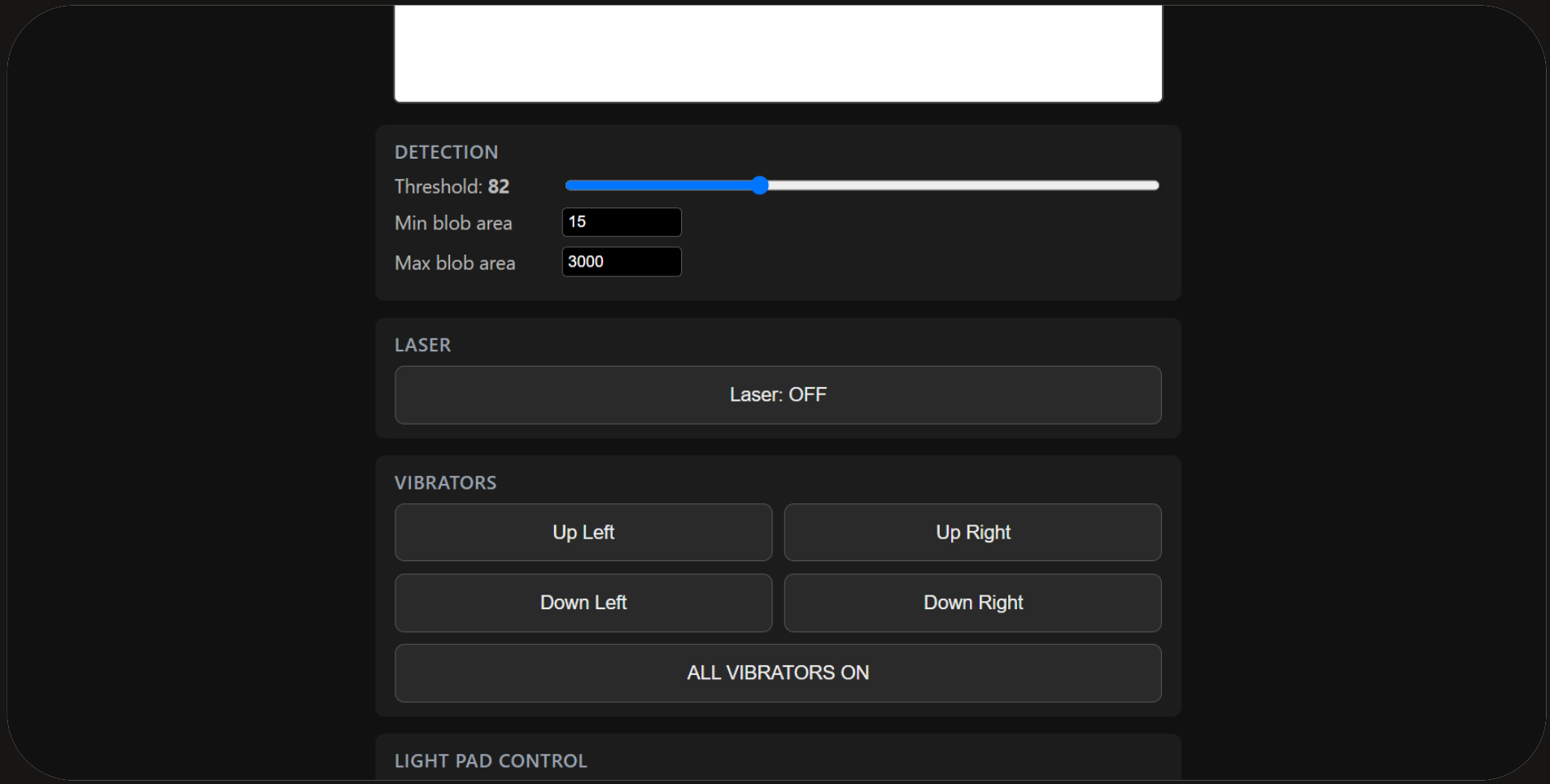
Software Implementation

COMPONENTS DETECTED

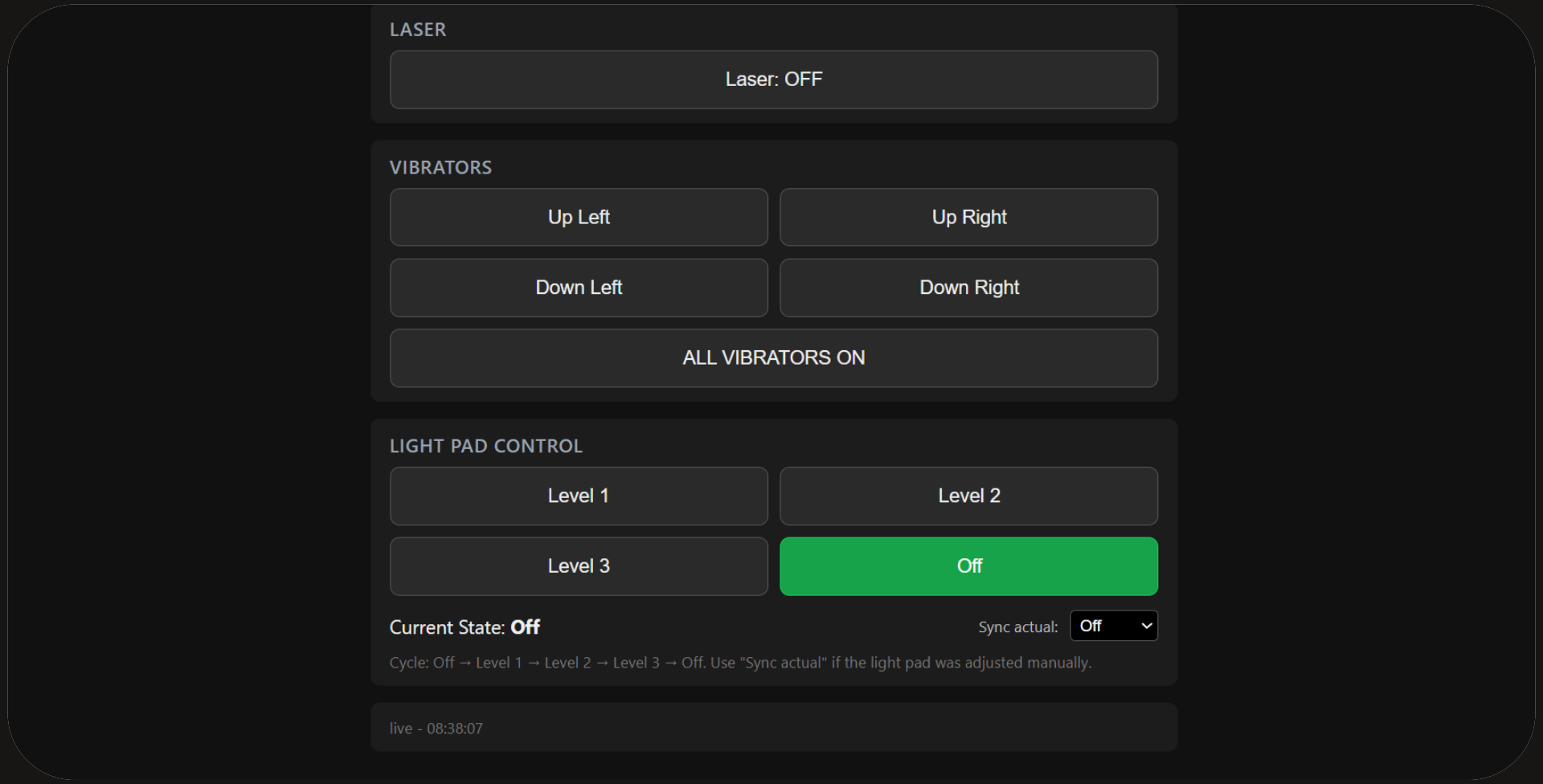
0



Software Implementation

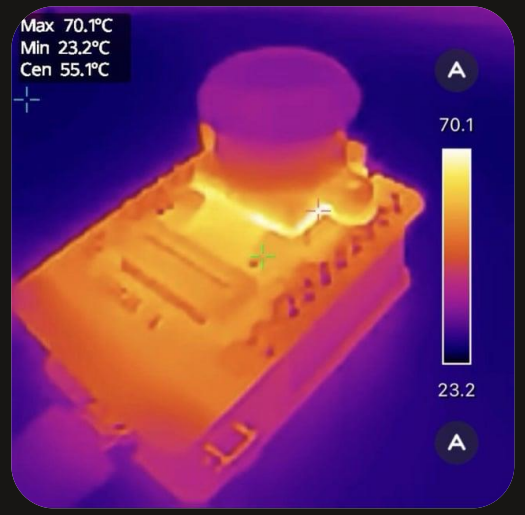


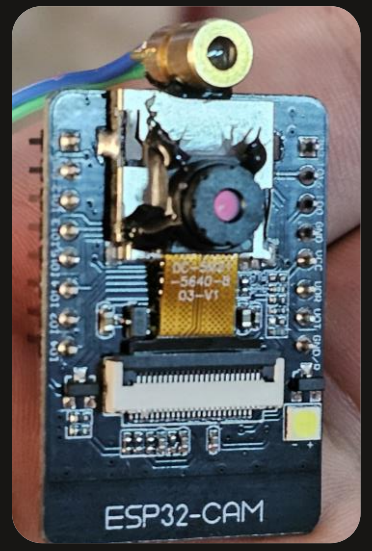
Software Implementation

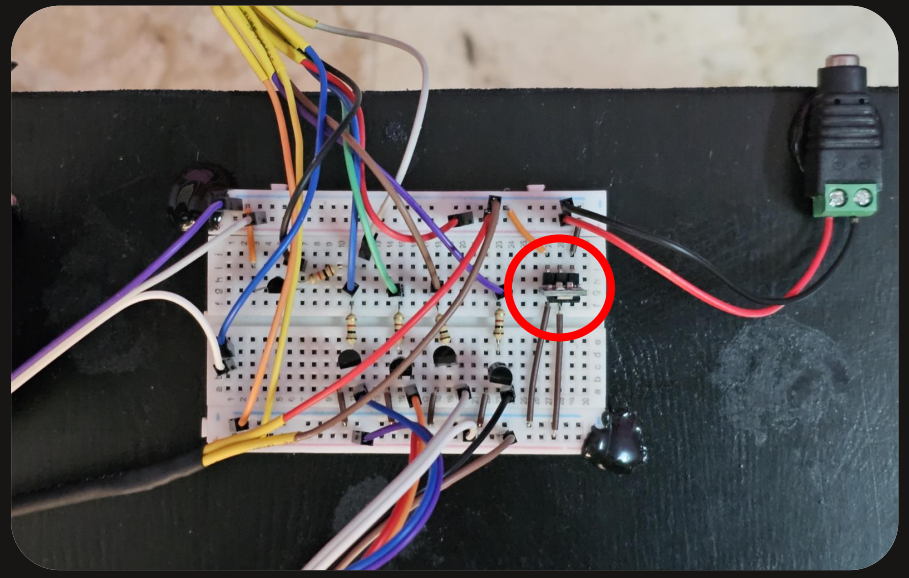


Challenges & Hardware Debugging

- **Thermal Management (ESP32 Overheating)**
  - Problem: Powering the ESP32-CAM via the 5V pin caused the onboard voltage regulator to severely overheat under the combined load of the camera and Wi-Fi, melting the silicon camera mount.
  - Solution: Bypassed the overburdened onboard regulator entirely by integrating a dedicated external 3.3V regulator, supplying stable power directly to the 3.3V pin and eliminating all thermal stress.

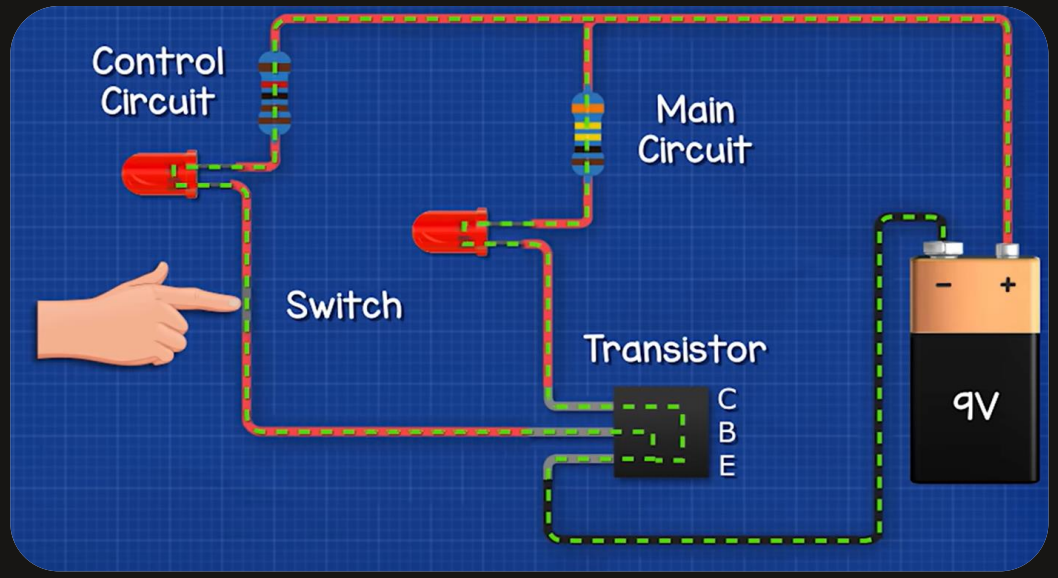


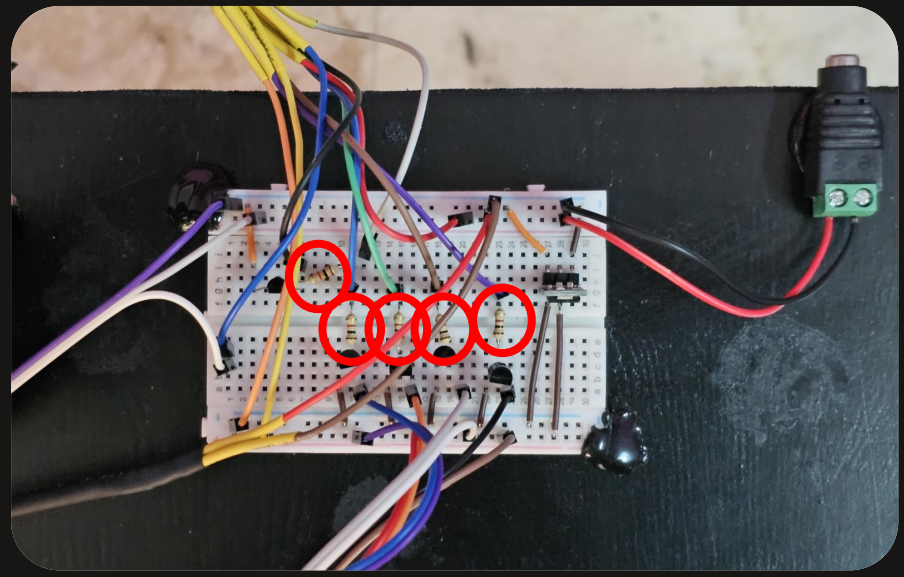




Challenges & Hardware Debugging

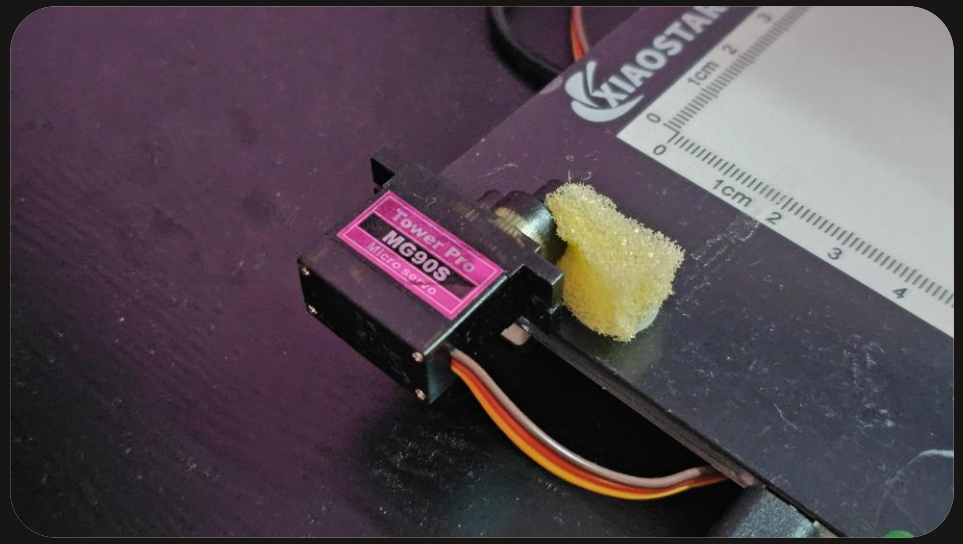
- **GPIO Current Protection (ESP32 Brownout Resets)**
  - Problem: Activating the vibration motors via the NPN transistors initially caused the ESP32 to instantly crash and reset
  - Solution: Identified that driving the transistors without base resistors was drawing excessive current directly from the GPIO pins. I installed current-limiting base resistors to protect the pins, resulting in stable logic-level switching without triggering brownout resets





Challenges & Hardware Debugging

- **IoT Integration of a Capacitive Interface**
  - Problem: I wanted to control the light pad via the web dashboard, but the commercial pad only featured a capacitive touch button with no digital data pins for the ESP32 to interface with.
  - Solution: Engineered an electromechanical bridge. I programmed a web-controlled servo motor equipped with a conductive tip (a water-dampened sponge) to successfully mimic human capacitance and physically actuate the button on command.



Future Improvements

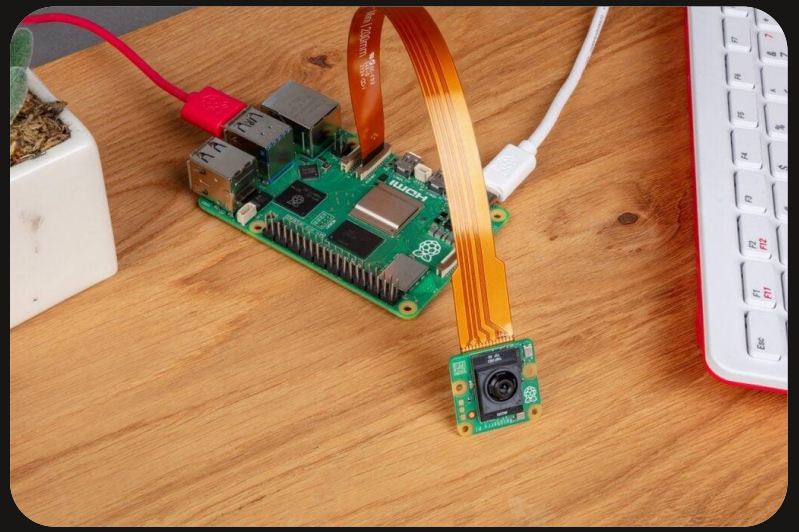
- **Integrated HMI Touch Display**
  - **Concept**: Upgrade from a web-based dashboard to a built-in physical touch screen, creating a fully standalone industrial workstation
  - **Current Constraint**: Budget limits and the ESP32-CAM's severe lack of free GPIO pins (most are consumed by the camera bus)
  - **Implementation:** Bypass I/O limits by integrating displays that use low-pin-count communication protocols, such as a UART intelligent display (e.g., Nextion) using only TX/RX, or an I2C touch screen using only SDA/SCL.

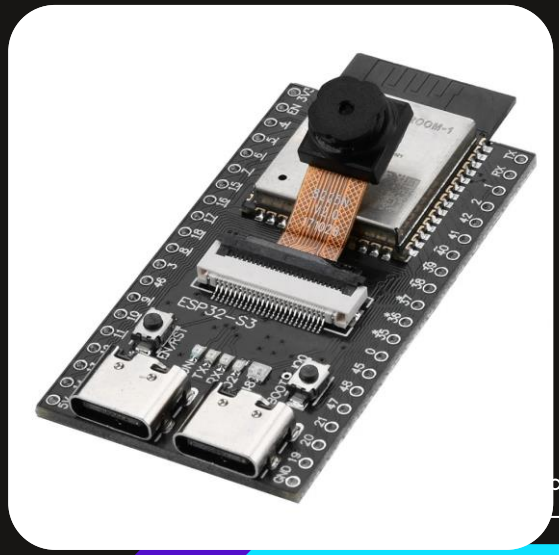




Future Improvements

- **Edge AI & Object Classification**
  - **Concept**: Transition from deterministic blob counting (BFS flood-fill) to neural network-based object detection.
  - **Implementation**: Train a lightweight machine learning model (using TinyML or TensorFlow Lite for Microcontrollers) to not only count the parts but accurately classify them by type (e.g., distinguishing a resistor from a ceramic capacitor)





Future Improvements

- **Solid-State Capacitive Switching**
  - **Concept**: Replace the electromechanical servo and wet sponge with a highly reliable, zero-moving-parts solution.
  - **Implementation**: Apply conductive copper tape directly over the light pad's capacitive sensor. By routing this tape through a digital relay or optocoupler to Ground (GND), the ESP32 can electrically simulate the capacitance of a human finger, eliminating mechanical wear and reducing power consumption.





Thanks